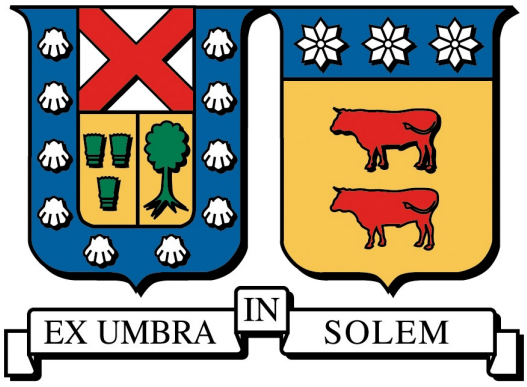


Sistema de Información para la Gestión

2do. Semestre 2026 / Programa Diurno / Campus Vitacura

Informe de Laboratorio 5

Autora:
Isidora Renata Córdova Stevenson
202204631-7, Ingeniería Civil Industrial

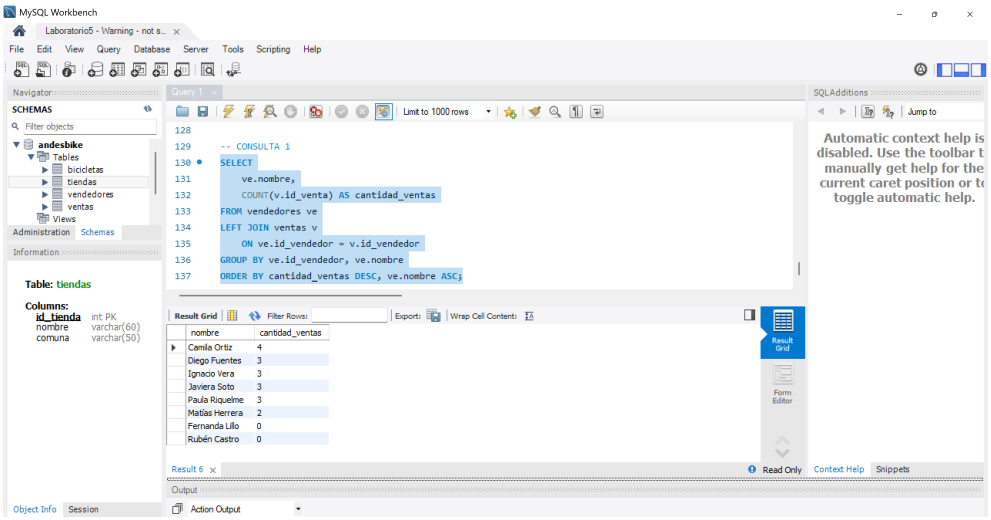
Profesores:
Patricio Cifuentes
José Luis Sáez T.

Enlace Github:

https://github.com/Isiusm/ICN292-Lab5-Cordova-Isidora

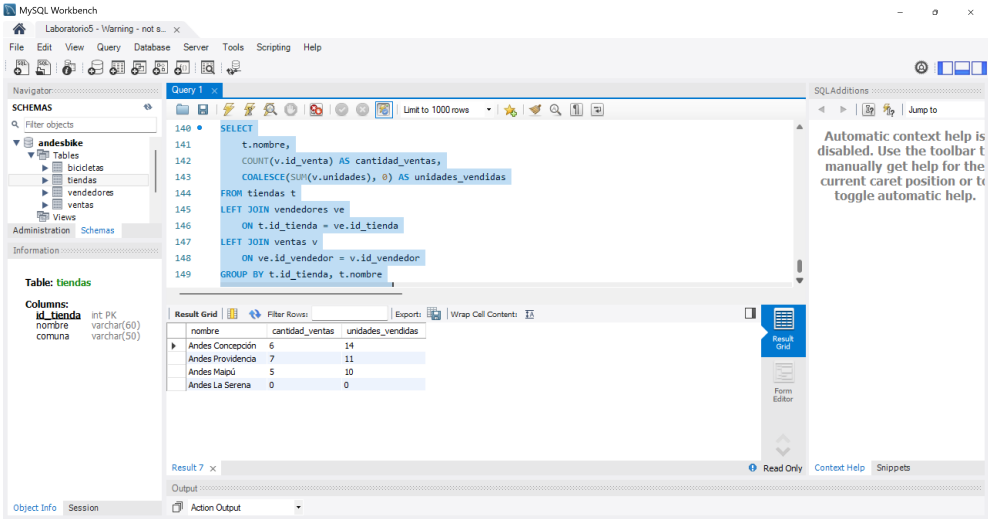
Consulta 1

```sql
SELECT ve.nombre,
       COUNT(v.id_venta) AS cantidad_ventas
FROM vendedores ve
LEFT JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
GROUP BY ve.id_vendedor, ve.nombre
ORDER BY cantidad_ventas DESC, ve.nombre ASC;
```



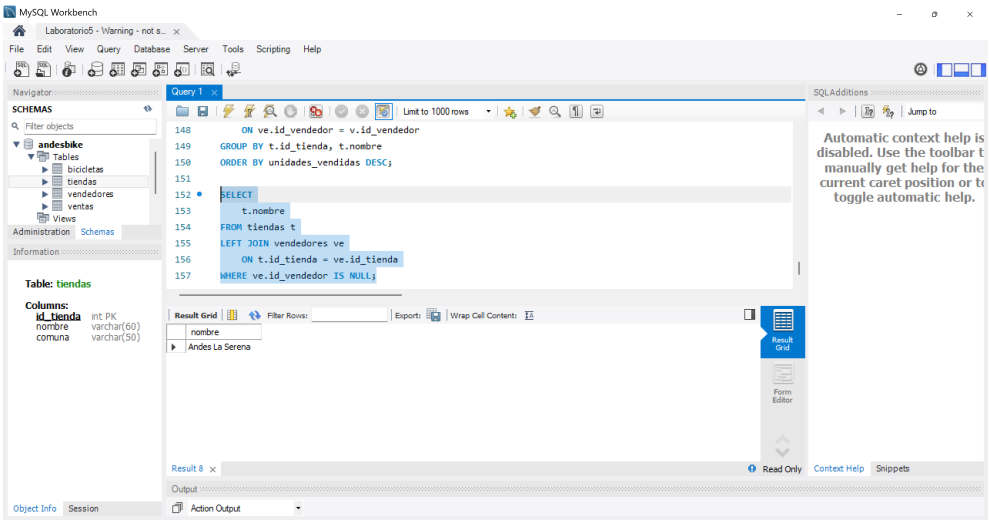
Consulta 2

```sql
SELECT t.nombre,
       COUNT(v.id_venta) AS cantidad_ventas,
       COALESCE(SUM(v.unidades), 0) AS unidades_vendidas
FROM tiendas t
LEFT JOIN vendedores ve
    ON t.id_tienda = ve.id_tienda
LEFT JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY unidades_vendidas DESC;
```



Consulta 3

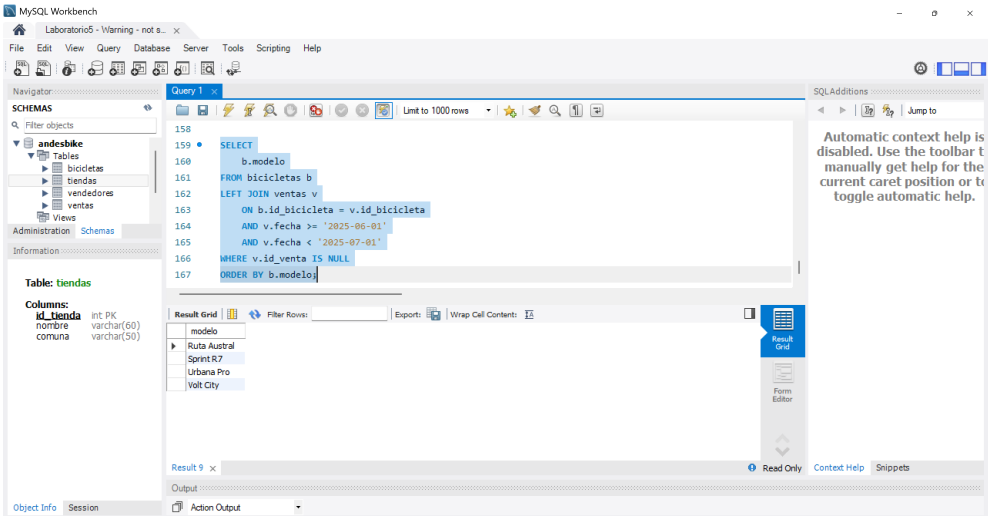
```sql
SELECT t.nombre
FROM tiendas t
LEFT JOIN vendedores ve
    ON t.id_tienda = ve.id_tienda
WHERE ve.id_vendedor IS NULL;
```



Consulta 4

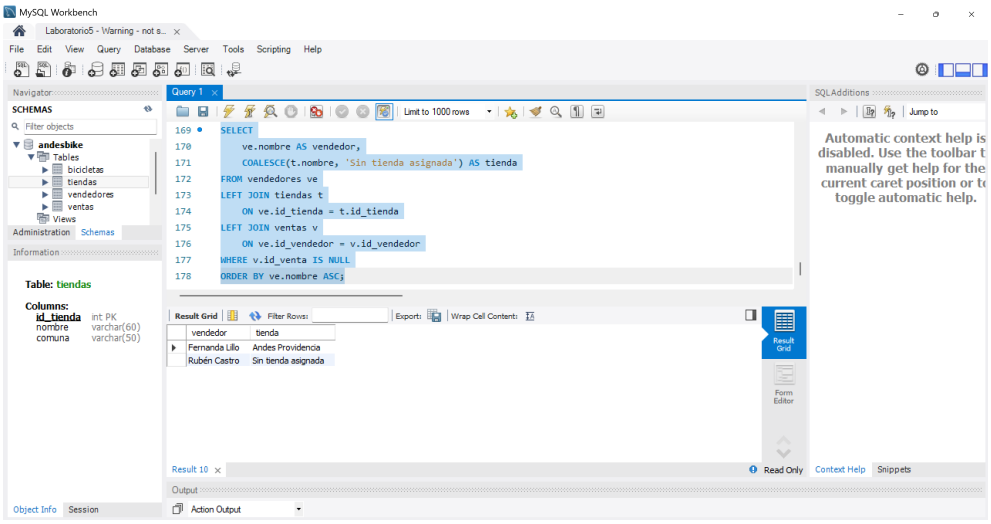
```sql
SELECT b.modelo
FROM bicicletas b
LEFT JOIN ventas v
    ON b.id_bicicleta = v.id_bicicleta
    AND v.fecha >= '2025-06-01'
    AND v.fecha < '2025-07-01'
WHERE v.id_venta IS NULL
ORDER BY b.modelo;
```

La condición de fecha se coloca en el `ON` para considerar únicamente las ventas de junio y conservar todas las bicicletas, permitiendo identificar aquellas que no tuvieron ventas durante ese período.



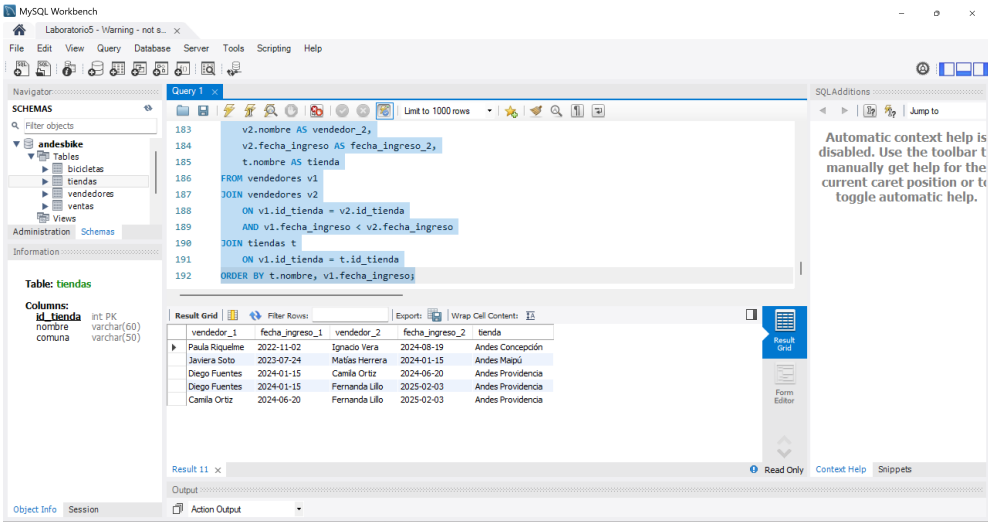
Consulta 5

```sql
SELECT ve.nombre AS vendedor,
       COALESCE(t.nombre, 'Sin tienda asignada') AS tienda
FROM vendedores ve
LEFT JOIN tiendas t
    ON ve.id_tienda = t.id_tienda
LEFT JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
WHERE v.id_venta IS NULL
ORDER BY ve.nombre ASC;
```



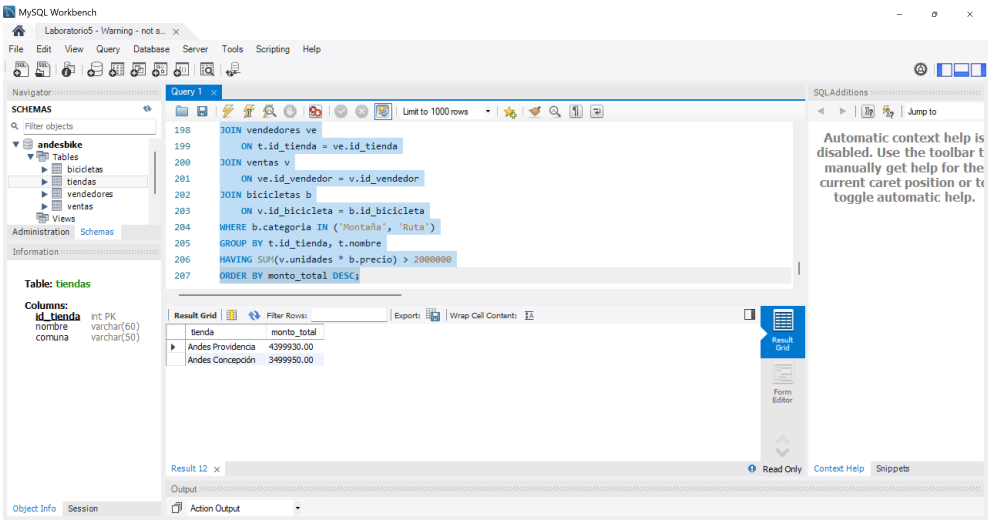
Consulta 6

```sql
SELECT v1.nombre AS vendedor_1,
       v1.fecha_ingreso AS fecha_ingreso_1,
       v2.nombre AS vendedor_2,
       v2.fecha_ingreso AS fecha_ingreso_2,
       t.nombre AS tienda
FROM vendedores v1
JOIN vendedores v2
    ON v1.id_tienda = v2.id_tienda
    AND v1.fecha_ingreso < v2.fecha_ingreso
JOIN tiendas t
    ON v1.id_tienda = t.id_tienda
ORDER BY t.nombre, v1.fecha_ingreso;
```



Consulta 7

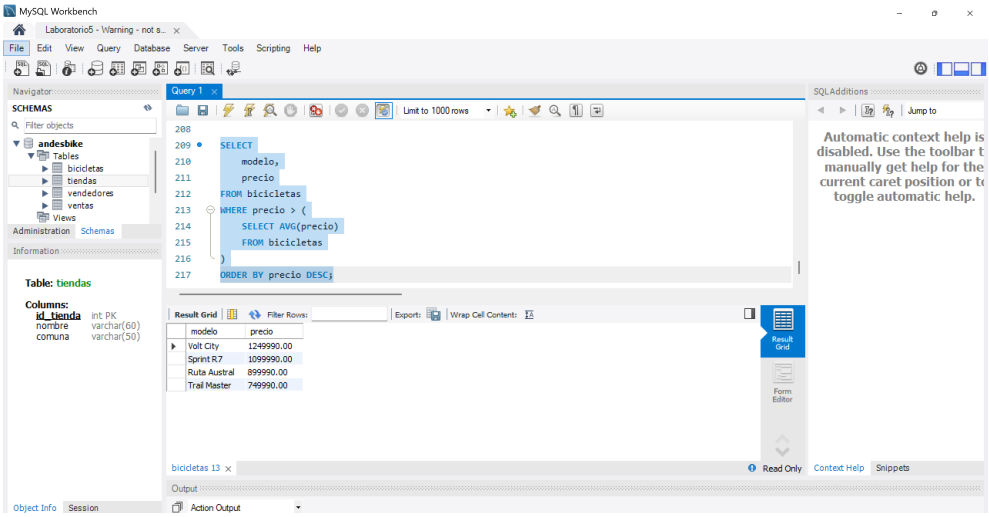
```sql
SELECT t.nombre AS tienda,
       SUM(v.unidades * b.precio) AS monto_total
FROM tiendas t
JOIN vendedores ve
    ON t.id_tienda = ve.id_tienda
JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
JOIN bicicletas b
    ON v.id_bicicleta = b.id_bicicleta
WHERE b.categoria IN ('Montaa', 'Ruta')
GROUP BY t.id_tienda, t.nombre
HAVING SUM(v.unidades * b.precio) > 2000000
ORDER BY monto_total DESC;
```



Consulta 8

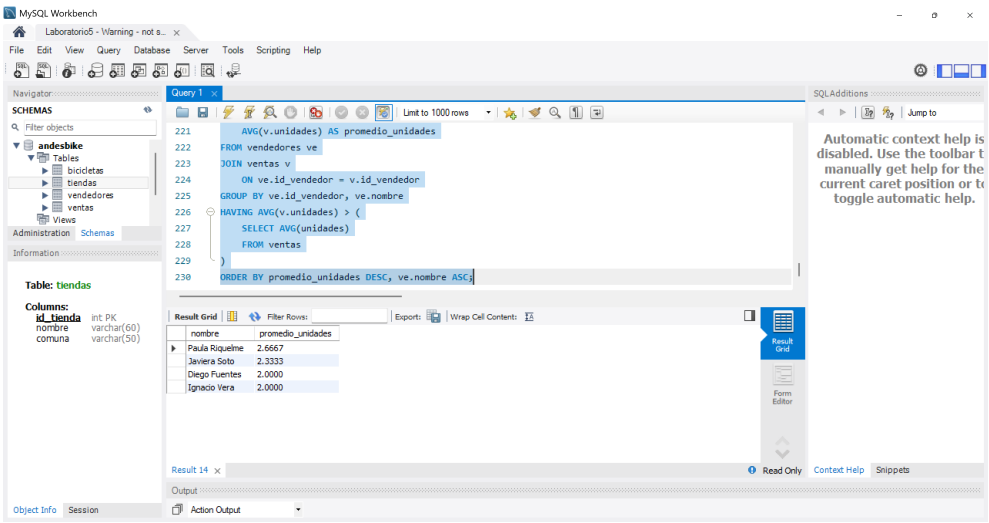
```sql
SELECT modelo,
       precio
FROM bicicletas
WHERE precio > (
    SELECT AVG(precio)
    FROM bicicletas
)
ORDER BY precio DESC;
```

La subconsulta calcula el precio promedio de todo el catálogo, cuyo valor es \$657.490, utilizando la función `AVG(precio)` sobre los registros de la tabla bicicletas. Posteriormente, se muestran los modelos cuyo precio supera dicho promedio.



Consulta 9

```sql
SELECT ve.nombre,
       AVG(v.unidades) AS promedio_unidades
FROM vendedores ve
JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
GROUP BY ve.id_vendedor, ve.nombre
HAVING AVG(v.unidades) > (
    SELECT AVG(unidades)
    FROM ventas
)
ORDER BY promedio_unidades DESC, ve.nombre ASC;
```



Consulta 10

```sql
SELECT ve.nombre AS vendedor,
       SUM(v.unidades * b.precio) AS monto_total
FROM vendedores ve
LEFT JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
LEFT JOIN bicicletas b
    ON v.id_bicicleta = b.id_bicicleta
GROUP BY ve.id_vendedor, ve.nombre
ORDER BY monto_total DESC, ve.nombre ASC;
```

Los vendedores sin ventas no muestran cero porque la función `SUM()` devuelve `NULL` cuando no existen registros asociados. Para que mostraran cero, habría que agregar `COALESCE(SUM(...), 0)`.

